

CSC343HY Practical 5

Aggregation in SQL & Relational Algebra

So far, every query you have written maps one input row to at most one output row. **Aggregation** breaks that rule: it collapses a set of rows into a single summary value. This practical covers the concept in two languages, relational algebra, where aggregation is expressed with the $\mathcal{G}$ operator, and SQL, where it is expressed with **GROUP BY** and **HAVING**, and then applies both to a practice problem.

1. Aggregation in Relational Algebra

The $\mathcal{G}$ operator

Standard relational algebra (σ , π , $\bowtie$, ...) has no built-in way to count rows or sum a column. We extend it with the **grouping-and-aggregation operator** $\mathcal{G}$, written:

$$g_1, g_2, \dots \mathcal{G}_{f_1(A_1) \text{ as } n_1, f_2(A_2) \text{ as } n_2, \dots} (R)$$

- The **subscript to the left** of $\mathcal{G}$ lists the *grouping attributes* $g_1, g_2, \dots$ the columns whose values define each group (corresponds to **GROUP BY**).
- The **subscript to the right** of $\mathcal{G}$ lists the *aggregate functions* f_i applied to column A_i , optionally renamed to n_i (corresponds to the aggregate expressions in **SELECT**).
- If there are **no grouping attributes**, $\mathcal{G}$ collapses the entire relation to a single row.

The supported aggregate functions are the same five as in SQL:

Function	Meaning
COUNT	Number of rows in the group (COUNT(*) includes nulls; COUNT(A) excludes nulls in A).
SUM	Sum of non-null values of a numeric attribute.
AVG	Mean of non-null values of a numeric attribute.
MAX	Largest value (works on numbers, strings, dates).
MIN	Smallest value.

Examples

Suppose we have a relation `loan(member_id, fine_amount)` with the rows from Tutorial 3.

Example 1. Total fines across all loans (no grouping):

$$\mathcal{G}_{\text{SUM}(\text{fine_amount}) \text{ as } \text{total_fines}} (\text{loan})$$

This collapses all rows to one, with a single attribute `total_fines`.

Example 2. Total fines *per member*:

$$\text{member_id } \mathcal{G}_{\text{SUM}(\text{fine_amount}) \text{ as } \text{total_fines}} (\text{loan})$$

One output row per distinct `member_id`, each carrying the sum of fines for that member.

Example 3. Members whose total fines exceed \$5 (adding a post-aggregation filter with σ):

$$\sigma_{total_fines > 5} \left(member_id \mathcal{G}_{SUM(fine_amount) \text{ as } total_fines} (loan) \right)$$

Because $\mathcal{G}$ produces an ordinary relation, you can wrap any relational algebra operator around it, including σ to filter groups (the RA equivalent of **HAVING**).

Reading $\mathcal{G}$ expressions. Evaluate from the inside out, just like any other relational algebra expression. Apply σ or π *before* $\mathcal{G}$ to filter rows before grouping; apply σ or π *after* $\mathcal{G}$ to filter or project the aggregated result.

Example 4. Among loans with a fine (**fine_amount** > 0), find each member’s loan count and maximum fine:

$$member_id \mathcal{G}_{COUNT(*) \text{ as } num_loans, MAX(fine_amount) \text{ as } max_fine} \left(\sigma_{fine_amount > 0} (loan) \right)$$

2. Aggregation in SQL

Execution order

SQL aggregation clauses execute in a fixed order. Reading your query in this order tells you exactly what each clause can “see”:

FROM $\longrightarrow$ **WHERE** $\longrightarrow$ **GROUP BY** $\longrightarrow$ **HAVING** $\longrightarrow$ **SELECT**

- **FROM** assembles the input relation (possibly a join).
- **WHERE** filters *individual rows* before any grouping. Aggregates are **not allowed** here.
- **GROUP BY** partitions surviving rows into groups by the listed columns.
- **HAVING** filters *groups* using aggregate conditions. This is the SQL equivalent of wrapping $\mathcal{G}$ in σ .
- **SELECT** computes the output columns. Every column listed must either be a grouping column or wrapped in an aggregate function.

SQL $\leftrightarrow$ Relational Algebra correspondence

SQL	Relational Algebra
GROUP BY g	Grouping attributes to the left of $\mathcal{G}$
Aggregate in SELECT	Aggregate function to the right of $\mathcal{G}$
WHERE condition	σ applied <i>before</i> $\mathcal{G}$
HAVING condition	σ applied <i>after</i> $\mathcal{G}$

The golden rule of GROUP BY. Every column in **SELECT** must either appear in **GROUP BY** or be wrapped in an aggregate. A column that is neither has no single value for the group, so PostgreSQL rejects the query.

3. Practice Problem

Work through the following problem by hand. First predict the result, then verify with a neighbour.

The table `q(r1 INTEGER, r2 INTEGER)` is created and populated as follows:

```
CREATE TABLE q (r1 INTEGER, r2 INTEGER);
INSERT INTO q VALUES (-1, -2); INSERT INTO q VALUES (1, 2);
INSERT INTO q VALUES (1, 2); INSERT INTO q VALUES (2, 4);
INSERT INTO q VALUES (2, 4); INSERT INTO q VALUES (3, 6);
INSERT INTO q VALUES (3, 1); INSERT INTO q VALUES (3, -2);
INSERT INTO q VALUES (3, 6); INSERT INTO q VALUES (3, 8);
INSERT INTO q VALUES (4, 1); INSERT INTO q VALUES (4, 0);
INSERT INTO q VALUES (4, -7); INSERT INTO q VALUES (5, 15);
INSERT INTO q VALUES (5, 10); INSERT INTO q VALUES (12, -11);
INSERT INTO q VALUES (12, 1); INSERT INTO q VALUES (12, -7);
INSERT INTO q VALUES (13, 1); INSERT INTO q VALUES (13, 3);
```

The query to evaluate:

```
SELECT r1, SUM(r2)
FROM q
WHERE r2 >= r1 - 5
GROUP BY r1
HAVING MIN(r2) >= 2;
```

Relational algebra equivalent

Before tracing through the SQL, write the equivalent $\mathcal{G}$ expression. Notice that the `WHERE` clause becomes a σ *before* $\mathcal{G}$, and the `HAVING` clause becomes a σ *after* $\mathcal{G}$:

$$\pi_{r1, total} \left(\sigma_{min_r2 \geq 2} \left(r1 \mathcal{G} \text{SUM}(r2) \text{ as total, MIN}(r2) \text{ as min_r2} \left(\sigma_{r2 \geq r1-5}(q) \right) \right) \right)$$

Read it inside-out: filter rows with $r2 \geq r1 - 5$, then group by $r1$ computing SUM and MIN, then keep only groups whose MIN is ≥ 2 , then project to the two output columns.

Step-by-step SQL trace

Step 1: `WHERE r2 >= r1 - 5`. For each row, check whether $r2 \geq r1 - 5$. Rows that fail are discarded before grouping.

r1	r2	r1 - 5	r2 ≥ r1-5?	Survives?
-1	-2	-6	-2 ≥ -6	✓
1	2	-4	2 ≥ -4	✓
1	2	-4	2 ≥ -4	✓
2	4	-3	4 ≥ -3	✓
2	4	-3	4 ≥ -3	✓
3	6	-2	6 ≥ -2	✓
3	1	-2	1 ≥ -2	✓
3	-2	-2	-2 ≥ -2	✓
3	6	-2	6 ≥ -2	✓
3	8	-2	8 ≥ -2	✓
4	1	-1	1 ≥ -1	✓
4	0	-1	0 ≥ -1	✓
4	-7	-1	-7 ≥ -1	×
5	15	0	15 ≥ 0	✓
5	10	0	10 ≥ 0	✓
12	-11	7	-11 ≥ 7	×
12	1	7	1 ≥ 7	×
12	-7	7	-7 ≥ 7	×
13	1	8	1 ≥ 8	×
13	3	8	3 ≥ 8	×

Step 2: **GROUP BY** r1. Collect surviving rows into groups:

r1	r2 values in group
-1	-2
1	2, 2
2	4, 4
3	6, 1, -2, 6, 8
4	1, 0
5	15, 10

Step 3: **HAVING** MIN(r2) >= 2. Compute MIN and SUM for each group, then discard groups where the minimum r2 is below 2:

r1	MIN(r2)	SUM(r2)	MIN ≥ 2?	Group survives?
-1	-2	-2	no	×
1	2	4	yes	✓
2	4	8	yes	✓
3	-2	19	no	×
4	0	1	no	×
5	10	25	yes	✓

Step 4: **SELECT** r1, SUM(r2). Output the two columns for the three surviving groups:

r1	sum
1	4
2	8
5	25

4. Exercises

Use the same table *q* as above for all questions. Write each answer in both SQL and relational algebra unless told otherwise.

1. Write a query (SQL + RA) that returns, for each *r1*, the *count* of rows and the *average* *r2*, but only for groups that contain **more than 2 rows**.
2. Write a query (SQL + RA) that returns the *r1* values where the *maximum* *r2* in the group is strictly greater than three times *r1*. (Hint: in SQL this condition goes in **HAVING**, not **WHERE**. Why?)
3. **SQL only.** Modify the query from Section 3 so that negative *r2* values are excluded *before* grouping (i.e. add a second **WHERE** condition). How does the result change, and why?
4. **Relational algebra only.** Express the query: “the overall minimum *r2* across the entire table” using $\mathcal{G}$ with no grouping attributes. Then extend it to find all rows whose *r2* equals that minimum (Hint: you will need $\bowtie$ and ρ).
5. **Trickier.** The query in Section 3 uses **WHERE** to filter rows *and* **HAVING** to filter groups. Could you move the **WHERE** condition into a **HAVING** clause instead and get the same result? Try it, predict what happens, and explain the difference.

5. Common Pitfalls

1. **Putting an aggregate in WHERE.** **WHERE** runs before grouping, so it cannot see aggregate values. Move aggregate conditions to **HAVING**.
2. **Selecting a non-grouped, non-aggregated column.** Every **SELECT** column must be in **GROUP BY** or wrapped in an aggregate; PostgreSQL will reject the query otherwise.
3. **Forgetting that WHERE filters rows, not groups.** A **WHERE** condition cannot refer to a group’s SUM or COUNT, those values do not exist yet.
4. **Confusing AVG denominator.** AVG divides the sum of non-null values by the *count of non-null values*, not by the total row count. Nulls are silently excluded.
5. **Applying σ in the wrong place in RA.** A σ before $\mathcal{G}$ filters rows (like **WHERE**); a σ after $\mathcal{G}$ filters groups (like **HAVING**). Swapping them produces a different, and usually wrong, result.

Before You Leave

Checklist

1. I can write the $\mathcal{G}$ operator with grouping attributes and aggregate functions.
2. I know that σ before $\mathcal{G}$ is **WHERE** and σ after $\mathcal{G}$ is **HAVING**.
3. I can trace a SQL query through **FROM** $\rightarrow$ **WHERE** $\rightarrow$ **GROUP BY** $\rightarrow$ **HAVING** $\rightarrow$ **SELECT** in order.
4. I know which columns are allowed in **SELECT** when **GROUP BY** is present.
5. I can work through the practice problem in Section 3 and explain why each group survives or is eliminated.